GLOWLOGICS

PROJECT SUBMISSION

CYBERSECURITY AND ETHICAL HACKING

PROJECT TITLE

DATA ENCRYPTION AND DECRYPTION TOOL

A Python-based application demonstrating secure message encryption and decryption

Project Submitted by:
SHIVAANI SHRE SR
Email: shivaani.050@gmail.com

Submitted for internship project evaluation

1. Abstract

The Data Encryption and Decryption Tool is a Python application that demonstrates how readable text can be converted into ciphertext and restored using a cryptographic key. This project uses the Fernet authenticated symmetric-encryption interface from the Python cryptography package. It is intended for learning and demonstration; the key must be protected because anyone who has it can decrypt data encrypted with it.

2. Objectives

- Understand confidentiality and the role of encryption in protecting information.
- Implement encryption and decryption using a well-established cryptography library.
- Generate and manage a key for the demonstration.
- Compare the practical workflow of encryption and decryption and explain basic security precautions.

3. Tools and Technologies

Technology	Purpose
Python 3	Implements the command-line application.
cryptography package	Provides Fernet authenticated symmetric encryption.
Fernet key	Encrypts and decrypts data; must be kept confidential.
Terminal	Accepts user input and displays demonstration results.

4. System Workflow

- **Key generation:** Generate a cryptographic key using the library.
- **Encryption:** Convert the input text to bytes and encrypt it with the key.
- **Display ciphertext:** Show the encrypted output, which is not readable as the original message.
- **Decryption:** Use the same key to restore the original message.
- **Error handling:** If the key is incorrect or the ciphertext has been modified, decryption fails.

5. Python Program

```
from cryptography.fernet import Fernet, InvalidToken

KEY_FILE = "secret.key"

def load_or_create_key():
    try:
        with open(KEY_FILE, "rb") as file:
            return file.read()
    except FileNotFoundError:
        key = Fernet.generate_key()
        with open(KEY_FILE, "wb") as file:
            file.write(key)
        return key

def main():
    key = load_or_create_key()
    cipher = Fernet(key)

    print("DATA ENCRYPTION AND DECRYPTION TOOL")
    print("Demo only: do not enter confidential information.")
    message = input("Enter a sample message: ").encode("utf-8")

    encrypted = cipher.encrypt(message)
    print("\\nEncrypted text:", encrypted.decode("utf-8"))

    try:
        decrypted = cipher.decrypt(encrypted).decode("utf-8")
        print("Decrypted text:", decrypted)
```

```
    except InvalidToken:
        print("Decryption failed: key is invalid or data was changed.")

    print("\\nKey file:", KEY_FILE)
    print("Keep secret.key private and do not share it.")

if __name__ == "__main__":
    main()
```

6. How to Execute

- Install Python 3 on your computer.
- Install the required library by running **python -m pip install cryptography**.
- Save the code as **encryption_tool.py**.
- Run **python encryption_tool.py** in the terminal.
- Enter a non-sensitive sample message. The program displays the encrypted text and then the decrypted original.
- The program creates **secret.key**. Keep this file private; if it is lost, data encrypted with that key cannot be recovered.

7. Sample Output

```
DATA ENCRYPTION AND DECRYPTION TOOL
Demo only: do not enter confidential information.
Enter a sample message: Hello Cybersecurity

Encrypted text: gAAAAA... (illustrative ciphertext)
Decrypted text: Hello Cybersecurity

Key file: secret.key
Keep secret.key private and do not share it.
```

The ciphertext shown above is shortened and illustrative. Actual ciphertext changes each time encryption runs.

8. Security Considerations

- Use established cryptographic libraries rather than designing a new encryption algorithm.
- Protect the key with suitable file permissions and never publish it with source code.
- Do not use this beginner demo as the sole protection for sensitive or production data.
- Avoid storing real passwords or personal information in a classroom demonstration.
- Use authenticated encryption so unauthorized changes to ciphertext are detected.

9. Algorithm Comparison

This implementation uses Fernet, which provides authenticated symmetric encryption: the same secret key is used for encryption and decryption. AES is a widely used symmetric cipher, while RSA is an asymmetric public-key algorithm commonly used for key exchange or digital signatures. For a practical application, hybrid designs often use symmetric encryption for data and public-key cryptography to protect the symmetric key. This project does not implement RSA directly.

10. Results and Discussion

The application demonstrates the full cycle of key creation, encryption, and decryption. The original message is recovered when the correct key and unmodified ciphertext are used. The program illustrates confidentiality and integrity protection while emphasizing that secure key handling is essential.

11. Limitations and Future Enhancements

Limitations: This is a command-line learning prototype, uses a local key file, and is not designed for production secret management or large-file processing.

Future enhancements: Add file encryption with careful streaming, a graphical interface, explicit key backup and rotation workflows, and a benchmark mode for comparing performance on non-sensitive test data.

12. Conclusion

The Data Encryption and Decryption Tool demonstrates how cryptographic libraries can protect message confidentiality and detect unauthorized changes. It provides practical experience with key generation, ciphertext, decryption, and the responsibility of keeping secret keys secure.

13. References

- Python documentation: https://docs.python.org/3/
- Python cryptography project documentation: https://cryptography.io/en/latest/fernet/
- NIST, Advanced Encryption Standard (AES): https://csrc.nist.gov/pubs/fips/197/final
- NIST, RSA and public-key cryptography resources: https://csrc.nist.gov/